Exact diagonal representability over real quadratic orders via weighted trace-form enumeration

TOMASZ KANIA*, Institute of Mathematics, Czech Academy of Sciences, Czech Republic and Institute of Mathematics, Jagiellonian University, Poland

`quadratic_diagonal` is a self-contained pure-Python reference package for exact constructive representability by diagonal quadratic lattices over maximal real quadratic orders. Given a squarefree radicand, totally positive diagonal coefficients, and a totally positive target, the software decides representability and returns an integral witness when one exists. The package implements weighted trace-form enumeration, dynamic-programming and meet-in-the-middle solvers, and batched bounded search for representable targets and bounded truants. The accompanying article proves the weighted discriminant identity, the distinct-value asymptotic, exact row-interval correctness, and pseudo-polynomial arithmetic-step bounds for fixed fields and fixed lattices. All comparisons in the core routines use integer trace and norm arithmetic; no floating-point boundary or ordering tests are used. The repository contains the installable package, non-interactive drivers, regression tests, validation sweeps, benchmark scripts, generated tables, performance plots, and an executed notebook. The implementation is intended both as a reusable research tool and as an executable specification that can be translated into compiled code or integrated into SageMath-style Python workflows.

CCS Concepts: • **Mathematics of computing** → **Mathematical software**; *Number theory*; *Algebraic algorithms*; • **Software and its engineering** → *Software libraries and repositories.*

Additional Key Words and Phrases: algorithms, quadratic forms, real quadratic fields, rings of integers, diagonal representability, dynamic programming, mathematical software

ACM Reference Format:
Tomasz Kania. 2026. Exact diagonal representability over real quadratic orders via weighted trace-form enumeration. *ACM Trans. Math. Softw.* 1, 1 (May 2026), 24 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

This paper presents a self-contained software package and accompanying exact algorithms for constructive representability by diagonal quadratic lattices over maximal real quadratic orders. The package, `quadratic_diagonal`, is intended as a transparent reference implementation: it exposes a small exact API, ships with regression tests, benchmark scripts, performance plots, and an executed notebook, and reproduces the tables and figures in Section 7 from the public repository source tree. Python is used deliberately as an executable specification language: the resulting code is easy to inspect, test, and translate, and it can be integrated directly into SageMath-style Python workflows even when higher-performance compiled backends are ultimately preferred.

The underlying mathematical problem is the following. Let

$$L = \langle a_1, \ldots, a_r \rangle, \qquad a_i \in O_D^+, \qquad \alpha \in O_D^+.$$

We want to decide whether

$$\alpha = a_1 x_1^2 + \cdots + a_r x_r^2 \qquad (x_1, \ldots, x_r \in O_D),$$

*IM CAS (RVO 67985840).

Author's Contact Information: Tomasz Kania, Institute of Mathematics, Czech Academy of Sciences, Prague, Czech Republic and Institute of Mathematics, Jagiellonian University, Kraków, Poland, tomasz.marcin.kania@gmail.com.

2026. Manuscript submitted to ACM

and, if so, to construct such a representation. The package API exposes an exact order model and routines for weighted enumeration, constructive dynamic programming, meet-in-the-middle search, and bounded-target batching; the README documents the precise entry-point names and command-line reproduction scripts.

From the literature viewpoint, this work sits between algorithmic quadratic-form computation over fields and arithmetic investigations over rings of integers. Koprowski and Czogała developed fundamental algorithms for quadratic forms over number fields, including isotropy, hyperbolicity, level, Pythagoras number, Witt equivalence, and related tasks [Koprowski and Czogała 2018]. Koprowski and Rothkegel later gave an algorithm for constructing anisotropic parts over number fields [Koprowski and Rothkegel 2023], while Koprowski subsequently provided explicit algorithms for sums of squares and for isotropic vectors over arbitrary global fields [Koprowski 2022, 2026]. In the real-quadratic integral setting, Raška studied when all elements of mO_D^+ are sums of squares and gave an efficient algorithm for that family-wise question [Raška 2023]. The present repository is organised in the same spirit as the author's companion public repository `quadratic_sos` [Kania 2026], which treats the unweighted $\langle 1, \ldots, 1 \rangle$ case. For broader arithmetic background on universal forms and indecomposables over totally real fields we refer to Kala's survey [Kala 2023]; recent work of Kala, Krásenský, and Romeo places criterion sets and diagonal universality in a general framework [Kala et al. 2024], and their interaction with sums of squares has become more explicit in work such as [Kala et al. 2025].

The novelty claim is deliberately narrow. We do *not* claim a new lattice-enumeration method in general. Rather, we specialise classical exact lattice enumeration to the weighted trace form over maximal real quadratic orders, make the resulting implementation fully exact and self-contained, integrate it with constructive diagonal representability, and strengthen the bounded-target layer by a batched dynamic program. The software contribution is as important here as the theorem statements: the package is installable, the public API mirrors the mathematical objects proved in the paper, the computational section is regenerated from scripts and notebook cells shipped in the repository, and we include a reproducible generic exact baseline in order to evaluate the specialised solvers against a direct value-product search.

The first main point is that the weighted trace form carries more structure than a generic box bound reveals. If

$$q_a(m + n\omega_D) = A_a m^2 + B_a mn + C_a n^2,$$

then its discriminant parameter is not merely positive; it is exactly

$$4A_aC_a - B_a^2 = 4\,\mathrm{Disc}(K_D)\,\mathrm{N}(a).$$

This identity gives the exact area of the trace ellipse and, after quotienting by the sign symmetry $x \sim -x$, the correct leading constant for the number of *distinct weighted values*. The second main point is algorithmic: the exact search can be organised over a canonical half-ellipse, so that the implementation visits exactly one representative from each non-zero ±-pair of roots and stores only one witness root per distinct value. The third main point is bounded search: for a trace bound B, all representable targets of trace at most B can be computed in one batched pass, rather than by rerunning the representability solver for each target separately.

Our main contributions are these.

(N1) We prove the discriminant identity for the weighted trace form, the resulting area formula for the trace ellipse, and the corresponding asymptotic for distinct weighted values of bounded trace.

(N2) We give an exact ellipse-sensitive weighted search which enumerates $\mathcal{S}_a(\alpha)$ by row intervals of the trace ellipse, scans only a canonical half-ellipse, and uses only integer arithmetic and exact comparisons in O_D.

(N3) We give exact constructive dynamic-programming and meet-in-the-middle algorithms for diagonal representability by $\langle a_1, \dots, a_r \rangle$, record permutation invariance, exploit internal coefficient reordering and repeated-coefficient caching in the implementation, and deduce a pseudo-polynomial bound for single targets via the dynamic-programming method.

(N4) We give an exact batched bounded representability algorithm and deduce a pseudo-polynomial $O(B^3)$ bound for bounded truants over a fixed field and a fixed diagonal lattice.

(N5) We provide a self-contained Python package, regression tests, benchmark tables, performance plots, and an executed notebook reporting meaningful structural quantities: distinct weighted values, canonical candidate counts, DP state sizes, half-sum sizes, coefficient-order effects, repeated-coefficient caching gains, and split preprocessing, combination, and end-to-end timings.

Software availability and reproducibility

The public repository associated with this paper will be maintained at https://github.com/tomaszkania/quadratic_diagonal; the submission bundle contains the same source tree. The top-level package `quadratic_diagonal` is installable with `pip install -e .` and exposes the exact API used throughout the paper. The repository also contains regression tests, a validation sweep, the script `scripts/reproduce_tables.py`, which regenerates all TSV tables in `data/` together with the performance plots in `paper/figures/`, and the executed notebook `notebooks/paper_illustrations.ipynb`. The structural columns in the computational section are deterministic; only the timing columns are machine-dependent.

For a TOMS Algorithm-paper submission the software component is supplied as a separate archive in the repository's `submission/` directory. That archive contains the installable source package, non-interactive driver programs, regression and validation tests, expected-output summaries, table and figure reproduction scripts, a manifest, and portability notes. The smoke-test driver `scripts/run_all_checks.py` requires no interactive input and is intended as the first command for referees; it exercises the public API, checks the regression suite, regenerates deterministic structural data, and records timing-dependent quantities separately.

The organisation is as follows. Section 2 recalls exact arithmetic in O_D and the trace-norm comparison lemma used in the implementation. Section 3 studies the weighted trace form, proves the discriminant identity, and derives the ellipse-area and lattice-point asymptotic formulas. Section 4 gives the exact weighted search on a canonical half-ellipse and its ellipse-sensitive complexity. Section 5 develops the constructive DP and meet-in-the-middle layers, records unit-square invariance, and proves the single-target pseudo-polynomial bound. Section 6 replaces per-target bounded search by a batched dynamic program and proves the bounded pseudo-polynomial complexity bound. Section 7 describes the software artefact, the reproducibility workflow, a generic exact baseline comparison, the benchmark data, the performance plots, and a short arithmetic case study. The degree-2 restriction is essential to the present geometry: over a totally real field of degree d the weighted trace form becomes a positive-definite lattice problem in rank d, so for cubic fields the search region is a genuine ellipsoid in $\mathbb{R}^3$ and the exact row-interval formulas used here no longer collapse to one-dimensional arithmetic.

2 Preliminaries

Let $D \geq 2$ be squarefree and write

$$K_D = \mathbb{Q}(\sqrt{D}), \qquad O_D = O_{K_D}.$$

We fix the standard integral generator

$$\omega_D = \begin{cases} \sqrt{D}, & D \equiv 2, 3 \pmod 4, \\ \dfrac{1+\sqrt{D}}{2}, & D \equiv 1 \pmod 4, \end{cases}$$

so that $\mathcal{O}_D = \mathbb{Z}[\omega_D]$. Every element of $\mathcal{O}_D$ is written uniquely as $a + b\omega_D$ with $a, b \in \mathbb{Z}$, and we identify it with the coefficient pair $(a, b) \in \mathbb{Z}^2$. We restrict throughout to the maximal order because this keeps both the coordinate model and the trace–norm comparison formulas uniform; extending the same exact algorithms to non-maximal orders would require additional bookkeeping for the conductor and the resulting coordinate constraints.

The two real embeddings are denoted by

$$\sigma_1, \sigma_2 : K_D \hookrightarrow \mathbb{R}, \qquad \sigma_1(\sqrt{D}) = \sqrt{D}, \qquad \sigma_2(\sqrt{D}) = -\sqrt{D}.$$

We write $x \preccurlyeq y$ if $\sigma_i(x) \le \sigma_i(y)$ for $i = 1, 2$, and $x \prec y$ if both inequalities are strict. Throughout, undecorated symbols such as $\le$ and $<$ refer to the usual order on $\mathbb{Z}$ or $\mathbb{R}$.

Lemma 2.1. *For every $z \in K_D$ one has:*

(i) $z \succcurlyeq 0$ *if and only if* $\mathrm{Tr}(z) \ge 0$ *and* $\mathrm{N}(z) \ge 0$*;*
(ii) $z \succ 0$ *if and only if* $\mathrm{Tr}(z) > 0$ *and* $\mathrm{N}(z) > 0$.

Proof. Write $z_i = \sigma_i(z)$. Then $\mathrm{Tr}(z) = z_1 + z_2$ and $\mathrm{N}(z) = z_1 z_2$. If $z_1, z_2 \ge 0$, then both quantities are non-negative. Conversely, z_1 and z_2 are the roots of

$$t^2 - \mathrm{Tr}(z)\, t + \mathrm{N}(z) = 0.$$

If $\mathrm{Tr}(z) \ge 0$ and $\mathrm{N}(z) \ge 0$, then the sum and product of the roots are both non-negative, so the two roots are either both non-negative or both non-positive; the non-negative sum rules out the second possibility. Hence $z_1, z_2 \ge 0$. This proves (i). The proof of (ii) is identical, with strict inequalities. □

Corollary 2.2. *For $x, y \in K_D$ one has*

$$x \preccurlyeq y \iff \mathrm{Tr}(y - x) \ge 0 \text{ and } \mathrm{N}(y - x) \ge 0.$$

Likewise,

$$x \prec y \iff \mathrm{Tr}(y - x) > 0 \text{ and } \mathrm{N}(y - x) > 0.$$

Proof. Apply Lemma 2.1 to $y - x$. □

Definition 2.3. For $x \in \mathcal{O}_D$, write $x \in \mathcal{O}_D^+$ if $x \succ 0$. A diagonal $\mathcal{O}_D$-lattice is an expression

$$L = \langle a_1, \ldots, a_r \rangle, \qquad a_i \in \mathcal{O}_D^+.$$

We say that $\alpha \in \mathcal{O}_D^+$ is represented by L, and write $\alpha \to L$, if there exist $x_1, \ldots, x_r \in \mathcal{O}_D$ such that

$$\alpha = a_1 x_1^2 + \cdots + a_r x_r^2.$$

If $D \equiv 1 \pmod 4$, then

$$\omega_D^2 = \omega_D + \frac{D-1}{4},$$

while for $D \equiv 2, 3 \pmod 4$ one has $\omega_D^2 = D$. The trace and norm of $a + b\omega_D$ are therefore

$$\mathrm{Tr}(a + b\omega_D) = \begin{cases} 2a + b, & D \equiv 1 \pmod 4, \\ 2a, & D \equiv 2, 3 \pmod 4, \end{cases}$$

and

$$\mathrm{N}(a + b\omega_D) = \begin{cases} a^2 + ab - \dfrac{D-1}{4} b^2, & D \equiv 1 \pmod 4, \\ a^2 - Db^2, & D \equiv 2, 3 \pmod 4. \end{cases}$$

3 Weighted trace forms and their geometry

Fix $a \in O_D^+$ and define the weighted trace form

$$q_a(x) = \mathrm{Tr}(ax^2) \qquad (x \in O_D).$$

Writing $x = m + n\omega_D$ with $m, n \in \mathbb{Z}$ yields an integral binary quadratic form

$$q_a(m + n\omega_D) = A_a m^2 + B_a mn + C_a n^2 \qquad (A_a, B_a, C_a \in \mathbb{Z}).$$

Proposition 3.1. *For every $a \in O_D^+$, the form q_a is a positive definite integral binary quadratic form on the lattice $O_D \cong \mathbb{Z}^2$.*

Proof. For every $x \in O_D$ we have $ax^2 \in O_D$, hence $q_a(x) = \mathrm{Tr}(ax^2) \in \mathbb{Z}$. If $x \neq 0$, then

$$q_a(x) = \sigma_1(a)\sigma_1(x)^2 + \sigma_2(a)\sigma_2(x)^2.$$

Since $a \in O_D^+$, both coefficients are strictly positive, and the two squares are non-negative and not both zero. Thus $q_a(x) > 0$ for every non-zero x. □

Definition 3.2. Let

$$\delta_a = 4A_aC_a - B_a^2.$$

Since q_a is positive definite, one has $A_a > 0$, $C_a > 0$, and $\delta_a > 0$.

Theorem 3.3 (Discriminant identity)**.** *Let* $\mathrm{Disc}(K_D)$ *denote the field discriminant of K_D. Then*

$$\delta_a = 4\,\mathrm{Disc}(K_D)\,\mathrm{N}(a).$$

Proof. Let $e_1 = 1$ and $e_2 = \omega_D$, and consider the symmetric bilinear form

$$T_a(x, y) = \mathrm{Tr}(axy) \qquad (x, y \in K_D).$$

Its Gram matrix in the basis (e_1, e_2) is

$$G_a = \big(T_a(e_i, e_j)\big)_{i,j=1}^2.$$

If

$$E = \begin{pmatrix} \sigma_1(e_1) & \sigma_1(e_2) \\ \sigma_2(e_1) & \sigma_2(e_2) \end{pmatrix},$$

then entrywise one has

$$G_a = E^\top \begin{pmatrix} \sigma_1(a) & 0 \\ 0 & \sigma_2(a) \end{pmatrix} E.$$

Indeed,

$$T_a(e_i, e_j) = \sigma_1(a)\sigma_1(e_i)\sigma_1(e_j) + \sigma_2(a)\sigma_2(e_i)\sigma_2(e_j).$$

Therefore

$$\det G_a = \sigma_1(a)\sigma_2(a)\,(\det E)^2 = \mathrm{N}(a)\,(\det E)^2.$$

For $a = 1$ this becomes the Gram matrix of the trace pairing on the integral basis $(1, \omega_D)$, so

$$(\det E)^2 = \det G_1 = \mathrm{Disc}(K_D).$$

Since $T_a(x, x) = \mathrm{Tr}(ax^2) = q_a(x)$, the diagonal entries of G_a are A_a and C_a, while the off-diagonal entry is $B_a/2$. On the other hand,

$$G_a = \begin{pmatrix} A_a & B_a/2 \\ B_a/2 & C_a \end{pmatrix},$$

whence

$$\det G_a = A_a C_a - \frac{B_a^2}{4} = \frac{\delta_a}{4}.$$

Combining the two determinant formulas yields the claim. □

Corollary 3.4 (Ellipse area)**.** *For $T > 0$, let*

$$\mathcal{E}_a(T) = \{(m, n) \in \mathbb{R}^2 : q_a(m + n\omega_D) \le T\}.$$

Then

$$\mathrm{area}(\mathcal{E}_a(T)) = \frac{\pi T}{\sqrt{\mathrm{Disc}(K_D)\,\mathrm{N}(a)}} = \frac{2\pi T}{\sqrt{\delta_a}}.$$

Proof. Let

$$M_a = \begin{pmatrix} A_a & B_a/2 \\ B_a/2 & C_a \end{pmatrix}.$$

Then $q_a(m + n\omega_D) = (m, n)M_a(m, n)^\top$. Since M_a is positive definite, the change of variables $y = M_a^{1/2}x$ sends $\mathcal{E}_a(T)$ to the Euclidean disk of radius $\sqrt{T}$. Thus

$$\mathrm{area}(\mathcal{E}_a(T)) = \frac{\pi T}{\sqrt{\det M_a}}.$$

Now $\det M_a = \delta_a/4$ and Theorem 3.3 gives $\delta_a = 4\,\mathrm{Disc}(K_D)\,\mathrm{N}(a)$. □

Corollary 3.5 (Candidate asymptotic)**.** *Let*

$$M_a(T) = \#\big(\mathcal{E}_a(T) \cap \mathbb{Z}^2\big).$$

Then

$$M_a(T) = \frac{\pi T}{\sqrt{\mathrm{Disc}(K_D)\,\mathrm{N}(a)}} + O_a(\sqrt{T}).$$

In particular, for fixed a one has $M_a(T) = \Theta_a(T)$.

Proof. A standard lattice-point estimate for bounded convex planar regions with smooth boundary gives

$$\#(\Lambda \cap C) = \frac{\mathrm{area}(C)}{\mathrm{covol}(\Lambda)} + O\big(\mathrm{perimeter}(C) + 1\big)$$

for each translate of a full-rank lattice $\Lambda \subseteq \mathbb{R}^2$; see, for instance, [Gruber and Lekkerkerker 1987, Chapter 2]. Here $\Lambda = \mathbb{Z}^2$ has covolume 1, and $\mathcal{E}_a(T)$ is an ellipse with perimeter $O_a(\sqrt{T})$. The claim now follows from Corollary 3.4. □

4 Exact weighted search

Fix $a, \alpha \in \mathcal{O}_D^+$ and set $T = \mathrm{Tr}(\alpha)$. Define the weighted root and value sets

$$\mathcal{R}_a(\alpha) = \{x \in \mathcal{O}_D \setminus \{0\} : ax^2 \preccurlyeq \alpha\}, \qquad \mathcal{S}_a(\alpha) = \{ax^2 : x \in \mathcal{R}_a(\alpha)\}.$$

Since $a \in \mathcal{O}_D^+$, every non-zero value ax^2 is automatically totally positive. We also define the canonical root set

$$\mathcal{R}_a^+(\alpha) = \{m + n\omega_D \in \mathcal{R}_a(\alpha) : n > 0 \text{ or } (n = 0,\ m > 0)\}.$$

Thus $\mathcal{R}_a^+(\alpha)$ contains exactly one representative from each non-zero sign pair. We call the elements of $\mathcal{R}_a^+(\alpha)$ the *canonical candidates* for the weighted search set.

Lemma 4.1. *If $a \in \mathcal{O}_D \setminus \{0\}$ and $x, y \in \mathcal{O}_D \setminus \{0\}$ satisfy $ax^2 = ay^2$, then $y = \pm x$. Consequently, the map $x \mapsto ax^2$ is exactly two-to-one on non-zero roots, and its restriction*

$$\mathcal{R}_a^+(\alpha) \to \mathcal{S}_a(\alpha)$$

is a bijection. In particular,

$$\#\mathcal{R}_a(\alpha) = 2\#\mathcal{S}_a(\alpha), \qquad \#\mathcal{R}_a^+(\alpha) = \#\mathcal{S}_a(\alpha).$$

Proof. Since $\mathcal{O}_D$ is an integral domain of characteristic 0, the equality $ax^2 = ay^2$ implies $x^2 = y^2$, hence $(x-y)(x+y) = 0$. Thus $y = \pm x$. Conversely, x and $-x$ clearly yield the same weighted square.

It remains to show that every non-zero sign pair $\{x, -x\}$ contains exactly one element of the canonical half. Write $x = m + n\omega_D$. If $n \neq 0$, then exactly one of n and $-n$ is positive. If $n = 0$, then $m \neq 0$, so exactly one of m and $-m$ is positive. Therefore $\{x, -x\}$ meets $\mathcal{R}_a^+(\alpha)$ in exactly one element. The bijectivity of the restricted map follows. □

Corollary 4.2 (Distinct-value asymptotic)**.** *For fixed $a \in \mathcal{O}_D^+$ and $B > 0$ one has*

$$\#\{ax^2 \in \mathcal{O}_D^+ : 0 < \mathrm{Tr}(ax^2) \le B\} = \frac{\pi B}{2\sqrt{\mathrm{Disc}(K_D)\,\mathrm{N}(a)}} + O_a(\sqrt{B}).$$

Proof. Let $M_a(B) = \#(\mathcal{E}_a(B) \cap \mathbb{Z}^2)$ as in Corollary 3.5. The lattice point $(0, 0)$ belongs to $\mathcal{E}_a(B)$, so $M_a(B) - 1$ is the number of non-zero roots satisfying $\mathrm{Tr}(ax^2) \le B$. By Lemma 4.1, the number of distinct weighted values is exactly $(M_a(B) - 1)/2$. The result follows from Corollary 3.5. □

Lemma 4.3. *If $x \in \mathcal{R}_a(\alpha)$, then $q_a(x) \le T$.*

Proof. If $x \in \mathcal{R}_a(\alpha)$, then $ax^2 \preccurlyeq \alpha$. Summing the two embedding inequalities gives

$$\mathrm{Tr}(ax^2) \le \mathrm{Tr}(\alpha) = T.$$

□

Proposition 4.4 (Exact row interval)**.** *Fix $n \in \mathbb{Z}$. Set*

$$\Delta_n = 4A_aT - \delta_a n^2, \qquad m_n^-(\alpha) = \left\lceil \frac{-B_a n - \lfloor\sqrt{\Delta_n}\rfloor}{2A_a} \right\rceil, \qquad m_n^+(\alpha) = \left\lfloor \frac{-B_a n + \lfloor\sqrt{\Delta_n}\rfloor}{2A_a} \right\rfloor,$$

whenever $\Delta_n \ge 0$. If $\Delta_n < 0$, then the row is empty. If $\Delta_n \ge 0$, then the integers m satisfying

$$A_a m^2 + B_a mn + C_a n^2 \le T$$

form the integer interval

$$I_n(\alpha) = \left[m_n^-(\alpha), m_n^+(\alpha)\right] \cap \mathbb{Z},$$

which may still be empty. In the case $\Delta_n \geq 0$, the row is non-empty if and only if $m_n^-(\alpha) \leq m_n^+(\alpha)$.

Proof. A direct calculation gives

$$4A_a\left(A_a m^2 + B_a mn + C_a n^2\right) = (2A_a m + B_a n)^2 + \delta_a n^2.$$

Hence the inequality is equivalent to

$$(2A_a m + B_a n)^2 \leq 4A_a T - \delta_a n^2 = \Delta_n.$$

If $\Delta_n < 0$, there is no solution. Otherwise set $s_n = \lfloor \sqrt{\Delta_n} \rfloor$. Since $2A_a m + B_a n$ is an integer, the inequality above is equivalent to

$$-s_n \leq 2A_a m + B_a n \leq s_n,$$

which yields the stated interval after division by $2A_a > 0$. The interval may still be empty if the ceiling endpoint exceeds the floor endpoint, and this is exactly the criterion $m_n^-(\alpha) \leq m_n^+(\alpha)$ for row non-emptiness when $\Delta_n \geq 0$. □

Remark 4.5. The condition $\Delta_n \geq 0$ is only a necessary discriminant condition for row non-emptiness. Even when $\Delta_n > 0$, the congruence class of $B_a n$ modulo $2A_a$ can prevent the admissible interval from containing an integer if the interval is too short.

Corollary 4.6. *If $I_n(\alpha) \neq \varnothing$, then*

$$|n| \leq \left\lfloor \sqrt{\frac{4A_a T}{\delta_a}} \right\rfloor.$$

Proof. If $I_n(\alpha)$ is non-empty, then $\Delta_n \geq 0$. Hence $\delta_a n^2 \leq 4A_a T$. □

Algorithm 1: ExactWeightedSearch(a, α)

Input: A totally positive coefficient $a \in O_D^+$ and target $\alpha \in O_D^+$.

Output: The distinct values $\mathcal{S}_a(\alpha)$ and one witness root for each value.

```
Compute the weighted trace form q_a(m + nω_D) = A_a m^2 + B_a mn + C_a n^2;
Set T ← Tr(α) and N ← ⌊√(4A_a T/δ_a)⌋;
Initialise an empty dictionary V from weighted values to roots;
for n = 0 to N do
    Compute the exact row interval I_n(α) from Proposition 4.4;
    if I_n(α) is empty then
        continue;
    Set J_n ← I_n(α);
    if n = 0 then
        replace J_n by J_n ∩ {1, 2, 3, ...};
    for m ∈ J_n do
        Set x ← m + nω_D and s ← ax^2;
        if s ≼ α then
            store the pair (s, x) in V if s is not present;
Return V;
```

Theorem 4.7 (Exact weighted search)**.** *Algorithm 1 returns exactly the weighted value set $\mathcal{S}_a(\alpha)$ together with one witness root for each value.*

Proof. Suppose the algorithm stores a value $s = ax^2$. Since $x \neq 0$ and $a \in O_D^+$, one has $s \succ 0$. The acceptance test is $s \preccurlyeq \alpha$, so $x \in \mathcal{R}_a(\alpha)$ and $s \in \mathcal{S}_a(\alpha)$.

Conversely, let $s \in \mathcal{S}_a(\alpha)$. Choose $x \in \mathcal{R}_a(\alpha)$ with $s = ax^2$, and let x^+ be the unique element of $\{x, -x\} \cap \mathcal{R}_a^+(\alpha)$ given by Lemma 4.1. Then $q_a(x^+) = q_a(x) \leq T$ by Lemma 4.3. Write $x^+ = m + n\omega_D$. Since x^+ is canonical, one has either $n > 0$ or $n = 0$ and $m > 0$. Corollary 4.6 shows that the algorithm reaches the row n, and Proposition 4.4 places m inside the exact row interval on that row. Thus the algorithm reaches x^+ and computes the same weighted value $s = a(x^+)^2 = ax^2$. Since $x^+ \in \mathcal{R}_a(\alpha)$, the acceptance test succeeds, so s is stored. □

Theorem 4.8 (Ellipse-sensitive complexity)**.** *Let*

$$M_a(\alpha) = \#\big(\mathcal{E}_a(\mathrm{Tr}(\alpha)) \cap \mathbb{Z}^2\big).$$

Then Algorithm 1 scans exactly

$$\frac{M_a(\alpha) - 1}{2}$$

canonical non-zero lattice points and performs

$$O\big(M_a(\alpha) + \sqrt{\mathrm{Tr}(\alpha)} + 1\big)$$

exact arithmetic steps, up to the cost of hash-table operations. It stores exactly $\#\mathcal{S}_a(\alpha)$ distinct weighted values and one witness root for each value. In particular, for fixed a the running time is $\Theta_a(\mathrm{Tr}(\alpha))$.

Proof. The algorithm scans precisely the rows with

$$0 \le n \le \left\lfloor \sqrt{\frac{4A_a \operatorname{Tr}(\alpha)}{\delta_a}} \right\rfloor,$$

so the number of row computations is $O(\sqrt{\operatorname{Tr}(\alpha)} + 1)$. On each scanned row, Proposition 4.4 computes the exact interval in constant-time arithmetic. The inner loop visits exactly the canonical non-zero lattice points in the trace ellipse $\mathcal{E}_a(\operatorname{Tr}(\alpha))$: for rows $n > 0$ it scans the full integer interval, while on the central row it keeps only positive abscissae. Every non-zero lattice point in the trace ellipse belongs to a unique $\pm$-pair, and every such pair has exactly one canonical representative. Since $(0, 0)$ is the only fixed point of the sign involution, the number of scanned candidates is $(M_a(\alpha) - 1)/2$. Since the weighted values are stored as integer coefficient pairs whose coordinates are bounded by $O(\operatorname{Tr}(\alpha))$, the hash-table insertions and lookups run in expected amortised constant time per operation. The final statement follows from Corollary 3.5. □

Remark 4.9. All complexity statements in the paper are arithmetic-step bounds for exact integer operations. In a bit-complexity model one must additionally account for the growth of the integer coordinates and of the trace and norm computations, which are polynomial in the input size for fixed field and fixed lattice.

Remark 4.10. Algorithm 1 is closely related to the Fincke–Pohst enumeration [Fincke and Pohst 1985] specialised to a rank-2 positive definite lattice. The additional features here are the canonical half-ellipse scan of Lemma 4.1, which halves the non-zero candidate count, and the exact order comparison in $\mathcal{O}_D$, which keeps the implementation free of floating-point boundary tests.

Remark 4.11. The same canonical row scan, with the final order-comparison filter removed, enumerates the trace-bounded value set

$$\{ax^2 \in \mathcal{O}_D^+ : 0 < \operatorname{Tr}(ax^2) \le B\}$$

in $O(B)$ time for fixed a.

5 Constructive diagonal representability

Fix a diagonal lattice

$$L = \langle a_1, \dots, a_r \rangle, \qquad a_i \in \mathcal{O}_D^+,$$

and a target $\alpha \in \mathcal{O}_D^+$. For each i set

$$V_i(\alpha) = \{0\} \cup \mathcal{S}_{a_i}(\alpha).$$

The implementation stores one witness root for each non-zero value in $\mathcal{S}_{a_i}(\alpha)$.

Proposition 5.1 (Additive reduction)**.** *One has*

$$\alpha \to L \iff \alpha \in V_1(\alpha) + \cdots + V_r(\alpha).$$

Proof. If

$$\alpha = a_1 x_1^2 + \cdots + a_r x_r^2,$$

then each summand is either 0 or belongs to $\mathcal{S}_{a_i}(\alpha)$, because every non-zero summand is totally positive and bounded above by α. Hence α lies in the displayed sumset. Conversely, if

$$\alpha = s_1 + \cdots + s_r, \qquad s_i \in V_i(\alpha),$$

then for each non-zero s_i choose the stored witness root x_i with $s_i = a_i x_i^2$, and set $x_i = 0$ when $s_i = 0$. Summing the resulting identities gives

$$\alpha = a_1 x_1^2 + \cdots + a_r x_r^2,$$

so $\alpha \to L$. □

Proposition 5.2 (Permutation invariance)**.** *Let π be a permutation of $\{1, \ldots, r\}$. Then*

$$\alpha \to \langle a_1, \ldots, a_r \rangle \iff \alpha \to \langle a_{\pi(1)}, \ldots, a_{\pi(r)} \rangle.$$

Proof. The displayed equality

$$\alpha = a_1 x_1^2 + \cdots + a_r x_r^2$$

holds if and only if the same summands are written in the permuted order

$$\alpha = a_{\pi(1)} x_{\pi(1)}^2 + \cdots + a_{\pi(r)} x_{\pi(r)}^2.$$

□

Remark 5.3. Proposition 5.2 justifies reordering the coefficients internally. In the implementation we sort the single-target value sets by non-decreasing $|\mathcal{S}_{a_i}(\alpha)|$ and the bounded value sets by non-decreasing $|\mathcal{W}_i(B)|$ before the combination phase. We also cache repeated coefficients so that each distinct weighted-value set is preprocessed only once.

Proposition 5.4 (Ordering upper bound)**.** *Let $s_i = \#\mathcal{S}_{a_i}(\alpha)$ and, for a permutation π of $\{1, \ldots, r\}$, define*

$$U(\pi) = \sum_{j=1}^{r} \prod_{k=1}^{j} \big(s_{\pi(k)} + 1\big).$$

Then $U(\pi)$ is minimised when the sequence $s_{\pi(1)}, \ldots, s_{\pi(r)}$ is ordered non-decreasingly.

Proof. It suffices to show that every adjacent inversion can be removed without increasing $U(\pi)$. Suppose that for some j one has

$$s_{\pi(j)} > s_{\pi(j+1)}.$$

Let π' be obtained from π by transposing $\pi(j)$ and $\pi(j+1)$. Then every prefix product of length $< j$ is unchanged. The prefix product of length j is multiplied by

$$\frac{s_{\pi(j+1)} + 1}{s_{\pi(j)} + 1} < 1,$$

so it strictly decreases. Every later prefix product contains both factors $s_{\pi(j)} + 1$ and $s_{\pi(j+1)} + 1$, hence is unchanged by the transposition. Thus $U(\pi') \le U(\pi)$, with strict inequality unless the two adjacent factors are equal. Iterating adjacent transpositions yields the non-decreasing ordering, which therefore minimises $U(\pi)$. □

Proposition 5.5 (Repeated-coefficient caching)**.** *Suppose that the coefficient list $a_1, \ldots, a_r$ assumes only u distinct values. With repeated-coefficient caching, the preprocessing stage for a single target α computes exactly u weighted search sets $\mathcal{S}_a(\alpha)$, and the preprocessing stage for bounded search up to trace bound B computes exactly u bounded weighted-value sets $\mathcal{W}_a(B)$. Consequently, for fixed field and fixed lattice, the preprocessing cost is*

$$O\big(u \operatorname{Tr}(\alpha)\big)$$

for single targets and

$$O(uB)$$

for bounded batched search.

Proof. Caching identifies equal coefficients and stores one exact weighted enumeration for each distinct coefficient value. Thus only u weighted enumerations are performed. By Theorem 4.8, each single-target weighted search costs $O(\mathrm{Tr}(\alpha))$ for fixed coefficient, and by Remark 4.11 each bounded weighted search costs $O(B)$ for fixed coefficient. Multiplying by the number u of distinct coefficients yields the stated bounds. □

Definition 5.6. Set $P_0(\alpha) = \{0\}$ and define recursively

$$P_j(\alpha) = \{y + s : y \in P_{j-1}(\alpha),\ s \in V_j(\alpha),\ y + s \preccurlyeq \alpha\} \qquad (1 \le j \le r).$$

Theorem 5.7 (Dynamic programming)**.** *For every j, the set $P_j(\alpha)$ is exactly the set of partial sums*

$$s_1 + \cdots + s_j, \qquad s_i \in V_i(\alpha), \qquad s_1 + \cdots + s_j \preccurlyeq \alpha.$$

Consequently,

$$\alpha \to L \iff \alpha \in P_r(\alpha).$$

Moreover, if one stores one predecessor pair (y, s) for each new element of $P_j(\alpha)$, then one can reconstruct a representation whenever $\alpha \in P_r(\alpha)$.

Proof. The statement for $j = 0$ is immediate. Assume it is true for $j - 1$. By definition, $P_j(\alpha)$ consists precisely of the sums of an element of $P_{j-1}(\alpha)$ and an element of $V_j(\alpha)$ whose total remains bounded by α. This is exactly the stated set of partial sums. The equivalence with representability now follows from Proposition 5.1. The predecessor pointers record one such decomposition at every stage, so backtracking from $\alpha \in P_r(\alpha)$ yields one tuple $(s_1, \ldots, s_r)$ and hence one representing root tuple. □

Corollary 5.8 (DP complexity and space)**.** *Let $s_j = \#\mathcal{S}_{a_j}(\alpha)$. Then the dynamic-programming algorithm runs in*

$$O\left(\sum_{j=1}^{r} \#P_{j-1}(\alpha)\,(s_j + 1)\right)$$

exact arithmetic steps. The constructive implementation which stores predecessor maps for backtracking uses

$$O\left(\sum_{j=1}^{r} \#P_j(\alpha)\right)$$

states, whereas a decision-only rolling-buffer variant uses

$$O\left(\max_{0 \le j \le r} \#P_j(\alpha)\right)$$

states.

Proof. At layer j, every state in $P_{j-1}(\alpha)$ is combined with every candidate in $V_j(\alpha)$, whose size is $s_j + 1$. Each accepted sum is inserted into a hash table at most once, which yields the time bound.

For the space bound, the constructive implementation stores one predecessor map for each layer $j = 1, \ldots, r$, and the jth predecessor map has exactly $\#P_j(\alpha)$ entries. Thus the total number of stored states is $\sum_{j=1}^{r} \#P_j(\alpha)$. If one only

wants the decision problem, the same recurrence can be run with two rolling buffers, storing only $P_{j-1}(\alpha)$ and $P_j(\alpha)$ at each step; this uses $O(\max_j \#P_j(\alpha))$ states. □

Definition 5.9. For $\alpha \in \mathcal{O}_D^+$, let

$$\mathcal{B}(\alpha) = \{\beta \in \mathcal{O}_D : 0 \preccurlyeq \beta \preccurlyeq \alpha\}.$$

Proposition 5.10. *Under the Minkowski embedding $\sigma : K_D \to \mathbb{R}^2$, the set $\mathcal{B}(\alpha)$ corresponds to the lattice $\sigma(\mathcal{O}_D)$ intersected with the rectangle*

$$[0, \sigma_1(\alpha)] \times [0, \sigma_2(\alpha)].$$

Consequently,

$$\#\mathcal{B}(\alpha) = \frac{\mathrm{N}(\alpha)}{\sqrt{\mathrm{Disc}(K_D)}} + O_D\big(\mathrm{Tr}(\alpha)\big).$$

Proof. By definition, $0 \preccurlyeq \beta \preccurlyeq \alpha$ if and only if both embedding coordinates of $\sigma(\beta)$ lie in the stated rectangle. That rectangle has area $\sigma_1(\alpha)\sigma_2(\alpha) = \mathrm{N}(\alpha)$ and perimeter $2\sigma_1(\alpha) + 2\sigma_2(\alpha) = 2\,\mathrm{Tr}(\alpha)$. Since $\sigma(\mathcal{O}_D)$ has covolume $\sqrt{\mathrm{Disc}(K_D)}$, the standard planar lattice-point estimate used in Corollary 3.5 yields the formula. □

Theorem 5.11 (Single-target pseudo-polynomial complexity)**.** *Fix the field K_D and the diagonal lattice $L = \langle a_1, \dots, a_r \rangle$. Then the constructive dynamic-programming algorithm decides whether $\alpha \to L$ and constructs a representation, when it exists, in*

$$O\big((\mathrm{N}(\alpha) + \mathrm{Tr}(\alpha))\,\mathrm{Tr}(\alpha)\big)$$

time and

$$O\big(\mathrm{N}(\alpha) + \mathrm{Tr}(\alpha)\big)$$

space. In particular, since $\mathrm{N}(\alpha) \le \mathrm{Tr}(\alpha)^2/4$ for totally positive α, the running time is $O(\mathrm{Tr}(\alpha)^3)$.

Proof. For each coefficient a_j, Corollary 4.2 gives

$$\#\mathcal{S}_{a_j}(\alpha) \le \#\{a_j x^2 \in \mathcal{O}_D^+ : 0 < \mathrm{Tr}(a_j x^2) \le \mathrm{Tr}(\alpha)\} = O\big(\mathrm{Tr}(\alpha)\big),$$

where the implied constant depends only on a_j. By Theorem 4.8, the exact weighted search computes each $\mathcal{S}_{a_j}(\alpha)$ in $O(\mathrm{Tr}(\alpha))$ time, so the total preprocessing cost is $O(\mathrm{Tr}(\alpha))$ for fixed r.

Every partial state in every $P_j(\alpha)$ lies in $\mathcal{B}(\alpha)$, hence

$$\#P_j(\alpha) \le \#\mathcal{B}(\alpha) = O\big(\mathrm{N}(\alpha) + \mathrm{Tr}(\alpha)\big)$$

by Proposition 5.10. Combining this with Corollary 5.8 and the bound $s_j = O(\mathrm{Tr}(\alpha))$ gives

$$\sum_{j=1}^{r} \#P_{j-1}(\alpha)(s_j + 1) = O\big((\mathrm{N}(\alpha) + \mathrm{Tr}(\alpha))\,\mathrm{Tr}(\alpha)\big)$$

for fixed r, which is the stated time bound. The constructive space usage is

$$O\left(\sum_{j=1}^{r} \#P_j(\alpha)\right) = O\big(\mathrm{N}(\alpha) + \mathrm{Tr}(\alpha)\big)$$

again because r is fixed.

Finally, if $u, v > 0$ with $u + v = \mathrm{Tr}(\alpha)$, then $uv \le (u+v)^2/4$, so $\mathrm{N}(\alpha) = \sigma_1(\alpha)\sigma_2(\alpha) \le \mathrm{Tr}(\alpha)^2/4$. □

Remark 5.12. The hidden constant in Theorem 5.11 depends on the rank r, on the coefficient list through the constants in Corollary 4.2, and on the field through $\mathrm{Disc}(K_D)^{-1/2}$ in Proposition 5.10. More explicitly, the preprocessing stage costs $O(u\ \mathrm{Tr}(\alpha))$ by Proposition 5.5, where u is the number of distinct coefficients, and each DP layer costs

$$O(\#\mathcal{B}(\alpha)\ \mathrm{Tr}(\alpha)) = O\left(\mathrm{Tr}(\alpha)\left(\frac{\mathrm{N}(\alpha)}{\sqrt{\mathrm{Disc}(K_D)}} + \mathrm{Tr}(\alpha)\right)\right).$$

Thus the constructive DP runtime is bounded by

$$O\left(r\ \mathrm{Tr}(\alpha)\left(\frac{\mathrm{N}(\alpha)}{\sqrt{\mathrm{Disc}(K_D)}} + \mathrm{Tr}(\alpha)\right)\right),$$

with the simpler $O(\mathrm{Tr}(\alpha)^3)$ form arising when K_D and L are fixed.

Proposition 5.13 (Meet-in-the-middle)**.** *Let $I \sqcup J = \{1, \dots, r\}$ be a partition. Define*

$$A_I(\alpha) = \left\{ \sum_{i \in I} s_i : s_i \in V_i(\alpha),\ \sum_{i \in I} s_i \preccurlyeq \alpha \right\}$$

and similarly $A_J(\alpha)$. Then

$$\alpha \to L \iff \exists u \in A_I(\alpha) \text{ such that } \alpha - u \in A_J(\alpha).$$

Thus, after the filtered half-sum tables are constructed, the final join takes linear time in the smaller half-table.

Proof. This is Proposition 5.1 with the summands grouped according to the partition $I \sqcup J$. □

Corollary 5.14 (MITM complexity and space)**.** *Fix an internal coefficient order and split it after the first t coefficients. For the left half set $L_0(\alpha) = \{0\}$ and define recursively*

$$L_j(\alpha) = \{y + s : y \in L_{j-1}(\alpha),\ s \in V_j(\alpha),\ y + s \preccurlyeq \alpha\} \qquad (1 \le j \le t).$$

For the right half set $R_0(\alpha) = \{0\}$ and define

$$R_j(\alpha) = \{y + s : y \in R_{j-1}(\alpha),\ s \in V_{t+j}(\alpha),\ y + s \preccurlyeq \alpha\} \qquad (1 \le j \le r - t).$$

Write $s_j = \#\mathcal{S}_{a_j}(\alpha)$. Then the constructive meet-in-the-middle algorithm runs in

$$O\left(\sum_{j=1}^{t} \#L_{j-1}(\alpha)\,(s_j + 1) + \sum_{j=1}^{r-t} \#R_{j-1}(\alpha)\,(s_{t+j} + 1) + \min\{\#L_t(\alpha), \#R_{r-t}(\alpha)\}\right)$$

exact arithmetic steps and stores

$$O\left(\sum_{j=1}^{t} \#L_j(\alpha) + \sum_{j=1}^{r-t} \#R_j(\alpha)\right)$$

states.

Proof. The two half tables are built by the same recurrence as in Theorem 5.7, applied to the left and right blocks separately. At stage j of the left half, every state in $L_{j-1}(\alpha)$ is combined with every value in $V_j(\alpha)$, whose size is $s_j + 1$, and likewise on the right. This gives the two summands in the time bound. Proposition 5.13 shows that after the half tables are built, the final join scans only the smaller half table and performs one hash lookup per state, which contributes the last summand.

For the space bound, the constructive implementation stores one predecessor table for each half-layer, and the jth predecessor table has exactly $\#L_j(\alpha)$ or $\#R_j(\alpha)$ entries. Summing over the layers gives the claim. □

Remark 5.15. The implementation splits after the first $\lfloor r/2 \rfloor$ coefficients in the internal order. For the modest ranks considered here this balanced split is adequate; optimising the split point by predicted half-sum sizes would be a natural next refinement.

Corollary 5.16 (Unit-square invariance)**.** *Let $\varepsilon \in \mathcal{O}_D^\times$. Then:*

(i) *$\alpha \to L$ if and only if $\varepsilon^2\alpha \to L$;*

(ii)

$$\alpha \to \langle a_1, \dots, a_r \rangle \iff \alpha \to \langle \varepsilon_1^2 a_1, \dots, \varepsilon_r^2 a_r \rangle$$

for all units $\varepsilon_1, \dots, \varepsilon_r \in \mathcal{O}_D^\times$.

Proof. If

$$\alpha = a_1 x_1^2 + \cdots + a_r x_r^2,$$

then

$$\varepsilon^2\alpha = a_1(\varepsilon x_1)^2 + \cdots + a_r(\varepsilon x_r)^2,$$

which proves (i); replacing ε by ε^{-1} gives the converse. The proof of (ii) is identical, using

$$(\varepsilon_i^2 a_i)(\varepsilon_i^{-1} x_i)^2 = a_i x_i^2.$$

□

Proposition 5.17 (Balanced unit-square normal form)**.** *Let $u \in \mathcal{O}_D^\times$ be a totally positive norm-one unit with $\sigma_1(u) > 1$. For every $a \in \mathcal{O}_D^+$ there exists $k \in \mathbb{Z}$ such that*

$$\sigma_1(u)^{-2} \le \frac{\sigma_1(u^{2k}a)}{\sigma_2(u^{2k}a)} < \sigma_1(u)^2.$$

Thus every totally positive coefficient is unit-square equivalent to a balanced one.

Proof. Set

$$\rho(a) = \log\left(\frac{\sigma_1(a)}{\sigma_2(a)}\right).$$

Since $\mathrm{N}(u) = 1$, one has $\sigma_2(u) = \sigma_1(u)^{-1}$ and therefore

$$\rho(u^{2k}a) = \rho(a) + 4k \log \sigma_1(u).$$

Choose $k \in \mathbb{Z}$ so that this quantity lies in the half-open interval $[-2\log\sigma_1(u), 2\log\sigma_1(u))$. Exponentiating gives the claim. □

Proposition 5.18 (Unit-square invariance of exact row complexity)**.** *Let $u \in \mathcal{O}_D^\times$ be a totally positive unit and let $a \in \mathcal{O}_D^+$. Then for every $T > 0$ one has*

$$M_{u^2 a}(T) = M_a(T).$$

Consequently the canonical half-ellipse scan visits the same number of non-zero root candidates for a and for $u^2 a$.

Proof. For every $x \in \mathcal{O}_D$,

$$q_{u^2 a}(x) = \mathrm{Tr}(u^2 a x^2) = \mathrm{Tr}\big(a(ux)^2\big) = q_a(ux).$$

Since multiplication by the unit u is a bijection of $\mathcal{O}_D$, the sets

$$\{x \in \mathcal{O}_D : q_{u^2 a}(x) \le T\} \qquad \text{and} \qquad \{y \in \mathcal{O}_D : q_a(y) \le T\}$$

have the same cardinality. This proves $M_{u^2a}(T) = M_a(T)$. The canonical half-scan keeps exactly one representative from each non-zero $\pm$-pair, so its candidate count is $(M_a(T) - 1)/2$ and is therefore invariant as well. □

Remark 5.19. The implementation now includes an exact balancing helper based on first-embedding sign tests. By Proposition 5.18, balancing does not change the exact row-candidate count, because that count is governed by the trace ellipse itself. Balancing is still useful for auxiliary box baselines: in $D = 5$ and trace bound $B = 100$, the coefficient $(89, 144)$ balances to $(1, 0)$; the exact row-candidate count stays equal to 70, while the sharp-box candidate count drops from 20849 to 97. The bundled notebook reproduces this example.

6 Bounded representability and bounded truants

For $B \geq 1$ define

$$\mathcal{P}_D(B) = \{\alpha \in \mathcal{O}_D^+ : \mathrm{Tr}(\alpha) \leq B\}.$$

This is the natural bounded target set for diagonal representability.

Proposition 6.1. *Under the Minkowski embedding*

$$\sigma : K_D \to \mathbb{R}^2, \qquad \sigma(x) = (\sigma_1(x), \sigma_2(x)),$$

the lattice $\sigma(\mathcal{O}_D)$ has covolume $\sqrt{\mathrm{Disc}(K_D)}$. Moreover,

$$\#\mathcal{P}_D(B) = \frac{B^2}{2\sqrt{\mathrm{Disc}(K_D)}} + O_D(B).$$

Proof. The covolume formula is standard for Minkowski embeddings of totally real number fields. The image of $\mathcal{P}_D(B)$ is the lattice $\sigma(\mathcal{O}_D)$ intersected with the open triangle

$$\mathcal{T}_B = \{(u, v) \in \mathbb{R}^2_{>0} : u + v \leq B\},$$

whose area is $B^2/2$ and perimeter is $O(B)$. Applying a standard perimeter-error lattice-point estimate for planar convex sets with piecewise smooth boundary, in particular for polygons, yields the formula. □

Definition 6.2. For each coefficient a_i define the bounded weighted-value set

$$\mathcal{W}_i(B) = \{0\} \cup \{a_i x^2 \in \mathcal{O}_D^+ : \mathrm{Tr}(a_i x^2) \leq B\}.$$

Set $\mathcal{Q}_0(B) = \{0\}$ and define recursively

$$\mathcal{Q}_j(B) = \{y + s : y \in \mathcal{Q}_{j-1}(B),\ s \in \mathcal{W}_j(B),\ y + s \in \{0\} \cup \mathcal{P}_D(B)\} \qquad (1 \leq j \leq r).$$

Algorithm 2: BatchedBoundedRepresentables(L, B)

Input: A diagonal lattice $L = \langle a_1, \dots, a_r \rangle$ and a trace bound $B \geq 1$.

Output: All non-zero represented targets of trace at most B.

```
For each i, compute W_i(B) by exact trace-ellipse enumeration;
Set Q_0(B) ← {0};
for j = 1 to r do
    Initialise Q_j(B) ← ∅;
    foreach y ∈ Q_{j−1}(B) do
        foreach s ∈ W_j(B) do
            Set t ← y + s;
            if t = 0 or t ∈ P_D(B) then
                insert t into Q_j(B);
Return Q_r(B) \ {0};
```

Theorem 6.3 (Exact batched bounded representability)**.** *For every j, the set $\mathcal{Q}_j(B)$ is exactly the set of sums*

$$s_1 + \cdots + s_j, \qquad s_i \in \mathcal{W}_i(B), \qquad s_1 + \cdots + s_j \in \{0\} \cup \mathcal{P}_D(B).$$

Consequently,

$$\mathcal{Q}_r(B) \setminus \{0\} = \{\alpha \in \mathcal{P}_D(B) : \alpha \to L\}.$$

In particular, the bounded truant set is

$$\mathrm{Tru}_B(L) = \mathcal{P}_D(B) \setminus \big(\mathcal{Q}_r(B) \setminus \{0\}\big).$$

The distinguished element $0 \in \mathcal{Q}_j(B)$ represents the case where all summands chosen so far are zero; it is needed both as the initial state and as a valid predecessor at every intermediate stage.

Proof. The induction on j is exactly the same as in Theorem 5.7. It therefore suffices to prove the final characterisation. If $\alpha \to L$ and $\mathrm{Tr}(\alpha) \leq B$, say

$$\alpha = a_1 x_1^2 + \cdots + a_r x_r^2,$$

then each non-zero summand is totally positive, so its trace is a positive integer. Since the sum of these traces is $\mathrm{Tr}(\alpha) \leq B$, every summand has trace at most B, hence belongs to the corresponding $\mathcal{W}_i(B)$. Therefore $\alpha \in \mathcal{Q}_r(B) \setminus \{0\}$.

Conversely, every element of $\mathcal{Q}_r(B) \setminus \{0\}$ is by construction a sum of one element from each $\mathcal{W}_i(B)$, and every non-zero element of $\mathcal{W}_i(B)$ has the form $a_i x_i^2$. Hence it is represented by L. □

Theorem 6.4 (Pseudo-polynomial complexity)**.** *Fix the field K_D and the diagonal lattice $L = \langle a_1, \dots, a_r \rangle$. Then the batched algorithm computes the bounded representable set and the bounded truant set of trace at most B in*

$$O(B^3)$$

time and

$$O(B^2)$$

space.

Proof. By Proposition 6.1, one has $\#\mathcal{P}_D(B) = O(B^2)$. For each fixed coefficient a_i, Corollary 4.2 gives

$$\#\mathcal{W}_i(B) = 1 + \frac{\pi B}{2\sqrt{\mathrm{Disc}(K_D)\,\mathrm{N}(a_i)}} + O_{a_i}(\sqrt{B}) = O(B).$$

By Remark 4.11, the exact canonical row scan computes each $\mathcal{W}_i(B)$ in $O(B)$ time, so the total preprocessing cost of building the $\mathcal{W}_i(B)$ is $O(B)$ for fixed r.

At stage j of the batched dynamic program, each state in $\mathcal{Q}_{j-1}(B)$ is combined with each value in $\mathcal{W}_j(B)$. Since $\#\mathcal{Q}_{j-1}(B) \le 1 + \#\mathcal{P}_D(B) = O(B^2)$ and $\#\mathcal{W}_j(B) = O(B)$, the work per stage is $O(B^3)$. For fixed r, the overall running time is therefore $O(B^3)$. The state sets are subsets of $\{0\} \cup \mathcal{P}_D(B)$, so the space usage is $O(B^2)$. □

Remark 6.5. The hidden constant in Theorem 6.4 depends on the rank r, on the number u of distinct coefficients through Proposition 5.5, and on the field through the factor $\mathrm{Disc}(K_D)^{-1/2}$ in Proposition 6.1. More explicitly, the preprocessing stage costs $O(uB)$, while each batched DP layer costs

$$O(\#\mathcal{P}_D(B)\,B) = O\left(B\left(\frac{B^2}{\sqrt{\mathrm{Disc}(K_D)}} + B\right)\right).$$

Thus the full bounded runtime is bounded by

$$O\left(r\,B\left(\frac{B^2}{\sqrt{\mathrm{Disc}(K_D)}} + B\right)\right),$$

with the simpler $O(B^3)$ form again arising when K_D and L are fixed.

Remark 6.6. Theorem 6.4 is the main algorithmic strengthening over the earlier per-target bounded-truant routine. The implementation below computes all bounded representables at once and then takes a set difference with $\mathcal{P}_D(B)$, rather than rerunning the diagonal representability solver for each target separately.

7 Software availability, reproducibility, and benchmark data

The accompanying public repository https://github.com/tomaszkania/quadratic_diagonal is the software artefact for this paper. It contains the installable package, exact arithmetic in $\mathcal{O}_D$, exact order comparison via Corollary 2.2, exact weighted trace-form enumeration, constructive DP and meet-in-the-middle representability, and the batched bounded-truant routine of Section 6. The code stores distinct weighted values by default, reorders the coefficients internally by increasing set size, caches repeated coefficients, and records the structural state sizes that actually drive the representability layer. The reproducibility workflow is intentionally simple: `python -m pytest -q` runs the regression suite, `python scripts/run_all_checks.py` runs the non-interactive TOMS smoke test, `python scripts/reproduce_tables.py` regenerates the TSV tables in `data/` together with the performance plots in `paper/figures/`, and the executed notebook reproduces the worked examples. The repository is organised in the same spirit as the companion public repository `quadratic_sos` [Kania 2026] for the unweighted case. The `submission/` directory contains the separated article PDF and software-component archive expected for a TOMS Algorithm-paper submission.

The empirical comparisons therefore focus on the right quantities.

(i) For weighted search, the meaningful baseline is the *sharp* box obtained from completing the square. We still list the deliberately crude box as a reference, but the real comparison is between the sharp box and exact row

D	a	α	$\lvert\mathcal{S}_a(\alpha)\rvert$	canon. row	canon. sharp	canon. crude	row ms	sharp ms	crude ms
21	$(2,1)$	$(30,0)$	13	19	73	6589	0.019	0.025	0.809
21	$(2,1)$	$(40,0)$	17	29	112	8950	0.028	0.043	1.126
6	$(3,-1)$	$(30,0)$	6	11	27	6937	0.010	0.011	0.831
5	$(1,1)$	$(30,0)$	27	41	123	2146	0.038	0.049	0.284
10	$(1,0)$	$(40,0)$	14	18	32	4105	0.014	0.016	0.444

Table 1. Weighted-search benchmarks. The structural columns report distinct weighted values and canonical candidate counts for exact row search, sharp box, and crude box. The meaningful empirical comparison is row versus sharp.

enumeration. The sharp box uses

$$|m| \le \left\lfloor \sqrt{4C_aT/\delta_a} \right\rfloor, \qquad |n| \le \left\lfloor \sqrt{4A_aT/\delta_a} \right\rfloor,$$

whereas the crude box drops the discriminant divisor and keeps only

$$|m| \le \lfloor \sqrt{4C_aT} \rfloor, \qquad |n| \le \lfloor \sqrt{4A_aT} \rfloor.$$

(ii) The weighted-search implementation scans only a canonical half-ellipse, so the candidate-count columns below report canonical candidates rather than all non-zero roots. The omitted non-zero root counts are recovered by doubling; the full trace-ellipse lattice-point count is recovered by doubling and adding the central point.

(iii) For diagonal representability, the relevant sizes are the distinct value counts $|\mathcal{S}_{a_i}(\alpha)|$ and the DP or half-sum state counts, not the raw number of roots.

(iv) The timing columns are now split honestly into preprocessing, combination, and end-to-end measurements. For the representability routines, preprocessing means weighted-set construction; for the bounded routines, preprocessing means building the sets $\mathcal{W}_i(B)$.

(v) For bounded truant search, the relevant comparison is between the full batched and full naive end-to-end routines. The batched preprocessing and batched combination times are reported separately only to explain where the speedup comes from.

All timings are machine-dependent; the structural columns are exact and reproducible. The present implementation is a transparent pure-Python reference implementation rather than a production-grade compiled lattice enumerator. In particular, systems such as Magma, PARI/GP, or SageMath would be expected to win in absolute wall time once their generic lattice-enumeration kernels and compiled arithmetic are brought to bear. The purpose of the benchmarks here is instead to validate the relative gains from the design choices of the note: ellipse-sensitive search, canonical half-ellipse enumeration, coefficient reordering, repeated-coefficient caching, batching, and the specialised combination layer. To keep the software artefact self-contained and directly reproducible, the paper therefore compares the specialised solvers against a bundled generic exact baseline that performs the same weighted-value preprocessing and then expands the full Cartesian product of the value lists.

Table 1 confirms two points. First, the exact row search always visits substantially fewer canonical candidates than the sharp box, and dramatically fewer than the deliberately crude box. Second, the table reports the quantity that actually matters algorithmically, namely the number of distinct weighted values. The total number of accepted roots is exactly twice this number by Lemma 4.1, but the representability layer does not need to store both signs.

Figure 1 visualises the same structural separation on a logarithmic scale. This is the software-facing interpretation of the ellipse-sensitive enumeration theorem: the exact row method improves the right candidate count, not merely the wall-clock time.

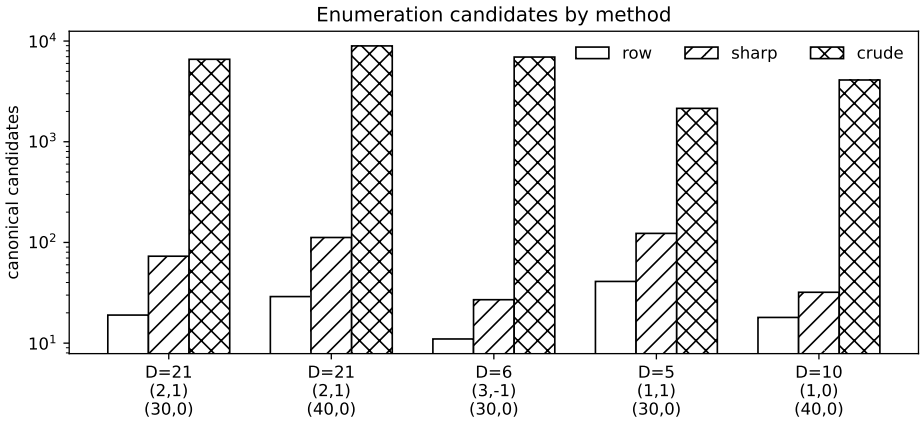

Fig. 1. Canonical candidate counts on a logarithmic scale for the weighted-search benchmarks of Table 1. The exact row search consistently stays below the sharp-box baseline and far below the deliberately crude box.

D	coeffs	α	ord	$\lvert S_{a_i}(\alpha)\rvert$	DP states	left	right	pre	DPc	DPt	MITMc	MITt
10	[(1, 0), (4, 1)]	(18, 2)	[1, 0]	[1, 5]	[2, 11]	2	6	0.023	0.007	0.030	0.006	0.029
	[(1, 0), (1, 0),		[0, 1,	[13, 13,	[14, 61,							
5	(1, 0)]	(15, 1)	2]	13]	108]	14	61	0.030	0.421	0.451	0.109	0.140
	[(2, 1), (1, 0),		[1, 2,	[12, 12,	[13, 60,							
21	(1, 0), (1, 0)]	(30, 0)	3, 0]	12, 13]	131, 195]	60	95	0.059	1.114	1.180	0.181	0.240
	[(1, 0), (2, 1),		[1, 2,	[5, 5,	[6, 15,							
13	(2, 1), (1, 0)]	(18, 0)	0, 3]	9, 9]	49, 84]	15	35	0.044	0.289	0.328	0.077	0.117

Table 2. Representability benchmarks. The "ord" column records the internal coefficient order used by the implementation; the timing columns are split into preprocessing, DP-combination, DP-total, meet-in-the-middle-combination, and meet-in-the-middle-total measurements.

D	α	ord	$\lvert \prod_i V_i(\alpha)\rvert$	pre	DPt	MITt	generic
10	$(18, 2)$	$[1, 0]$	12	0.027	0.034	0.034	0.031
5	$(15, 1)$	$[0, 1, 2]$	2744	0.034	0.481	0.151	0.717
21	$(30, 0)$	$[1, 2, 3, 0]$	30758	0.068	1.259	0.242	9.727
13	$(18, 0)$	$[1, 2, 0, 3]$	3600	0.046	0.343	0.127	1.203

Table 3. Surrogate general-purpose exact baseline for single-target representability. The "generic" column expands the full Cartesian product of the value lists after the same exact preprocessing used by DP and meet-in-the-middle.

Table 2 makes the timing scope explicit. The preprocessing time is shared weighted-set construction. The "DPc" and "MITMc" entries measure only the combination layers, whereas "DPt" and "MITt" are the full end-to-end routines. This is why the meet-in-the-middle speedup in the last three examples remains visible even after the shared preprocessing is included.

Table 3 is the TOMS-facing comparison point. The artefact does not depend on an external CAS, so instead of timing SageMath or PARI/GP directly we bundle a reproducible generic exact baseline that mirrors the post-enumeration task a general-purpose system would still have to solve: given the exact weighted-value sets, search their full Cartesian product for a representation. This keeps the comparison portable while still showing the algorithmic advantage of reusing partial sums. The structural column $\lvert \prod_i V_i(\alpha)\rvert$ records the raw search size of that baseline.

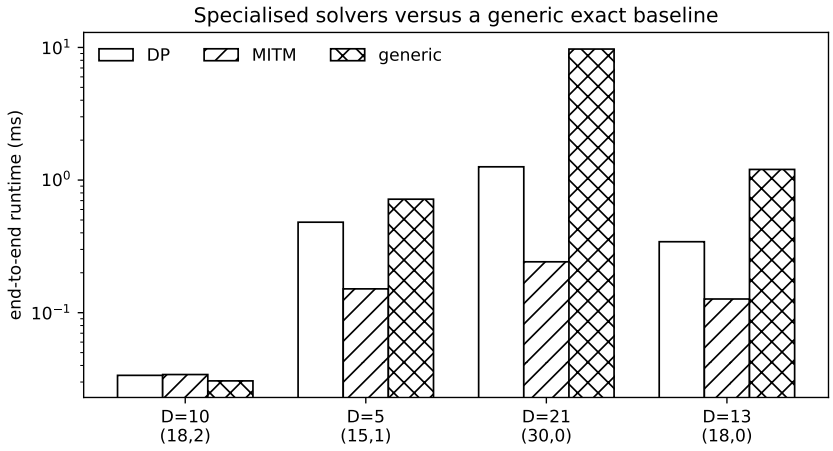

Fig. 2. End-to-end runtimes for constructive DP, meet-in-the-middle, and the bundled generic exact baseline of Table 3. The specialised solvers exploit partial-sum reuse, whereas the generic baseline expands the full Cartesian product of the value lists.

D	coefficients	α	ord	original states	optimised states	pre_{unc}	pre_{c}	DP_{orig}	DP_{opt}
10	$[(1,0),(4,1)]$ $[(1,0),(2,1),$	$(18,2)$	$[1,0]$ $[1,2,$	$[6,11]$ $[10,31,$	$[2,11]$ $[6,15,$	0.020	0.019	0.029	0.027
13	$(2,1),(1,0)]$ $[(2,1),(1,0),$	$(18,0)$	$0,3]$ $[1,2,$	$49,84]$ $[14,95,$	$49,84]$ $[13,60,$	0.083	0.033	0.365	0.313
21	$(1,0),(1,0)]$	$(30,0)$	$3,0]$	$190,195]$	$131,195]$	0.088	0.046	1.499	1.038

Table 4. Coefficient-order and repeated-coefficient optimisation. The timing columns record, respectively, uncached preprocessing, cached preprocessing, original end-to-end DP runtime, and optimised end-to-end DP runtime.

As a concrete constructive witness, for $D = 5$, $L = \langle (1,0), (1,0), (1,0) \rangle$, and $\alpha = (15,1)$, the dynamic program returns the root tuple

$$\big((0,1), (-1,2), (3,0)\big),$$

hence the explicit representation

$$(15,1) = (1,0)(0,1)^2 + (1,0)(-1,2)^2 + (1,0)(3,0)^2 = (1,1) + (5,0) + (9,0).$$

This illustrates that the implementation is genuinely constructive: it returns actual roots, not merely a yes/no answer.

Table 4 illustrates the practical value of Proposition 5.4 and Proposition 5.5. Sorting by non-decreasing weighted-set size reduces the intermediate DP upper bound and, in practice, the DP state counts as well, while repeated-coefficient caching reduces preprocessing when a coefficient occurs more than once. The second row is typical: the two repeated coefficients $(2,1)$ are preprocessed only once, and the internal ordering cuts the first two DP layers from $[10,31]$ to $[6,15]$.

Table 5 illustrates the algorithmic gain from Theorem 6.4. The structural columns show that the batched routine works with the bounded value sets $\mathcal{W}_i(B)$ and the intermediate bounded state sets $\mathcal{Q}_j(B)$ exactly as the proof predicts. The timing column shows the important comparison: the full batched end-to-end routine is already substantially faster than solving each target separately, even at these modest bounds.

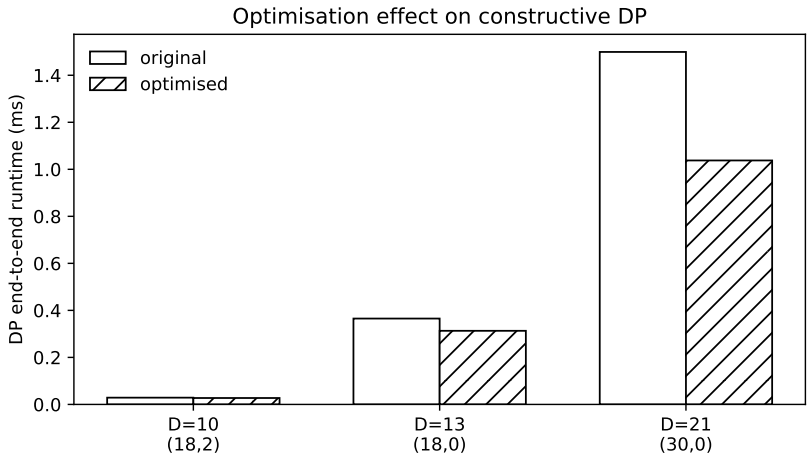

Fig. 3. End-to-end constructive DP runtimes before and after internal coefficient reordering and repeated-coefficient caching for the optimisation benchmarks of Table 4.

D	coefficients	B	represented	truants	$\lvert\mathcal{W}_{\tilde{t}}(B)\rvert - 1$	batched states	times (ms)
5	$[(1,0),(1,0),(1,0)]$	10	25	0	$[7,7,7]$	$[8,20,26]$	0.010/0.065/0.112/0.685
10	$[(1,0),(4,1)]$	18	6	21	$[1,3]$	$[2,7]$	0.011/0.004/0.043/0.604
21	$[(2,1),(1,0),(1,0)]$	16	23	7	$[4,6,6]$	$[5,18,24]$	0.016/0.047/0.105/0.998

Table 5. Batched bounded search versus per-target recomputation. The timing column lists, in order, batched preprocessing, batched dynamic-programming combination, full batched end-to-end runtime, and full naive end-to-end runtime.

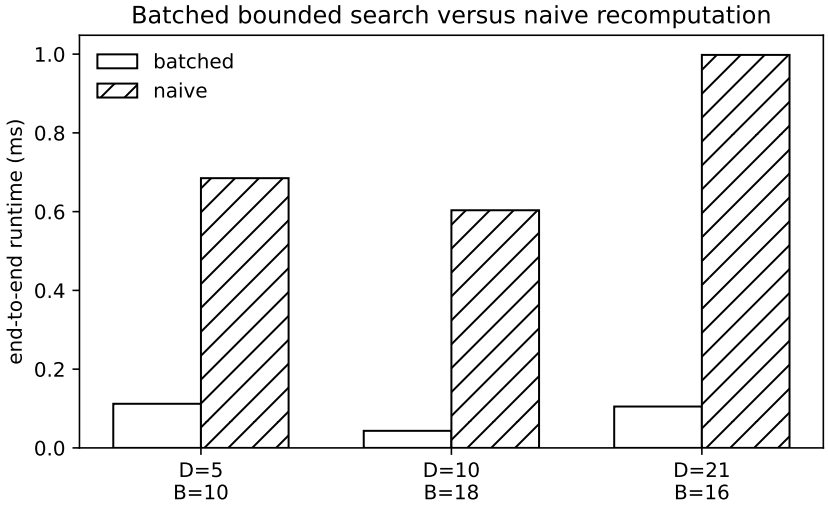

Fig. 4. End-to-end runtimes for the batched bounded routine and naive per-target recomputation in Table 5. The important comparison is the full batched end-to-end runtime versus the full naive end-to-end runtime.

D	coefficients	B	bounded truant set
5	$[(1,0),(1,0),(1,0)]$	12	none
5	$[(1,0),(1,0),(2,1)]$	40	none
13	$[(1,0),(2,1),(2,1),(1,0)]$	18	$[(3,-1),(3,0),(3,2),(6,-1),(6,3)]$
21	$[(2,1),(1,0),(1,0)]$	16	$[(4,2),(6,-2),(5,2),(6,0),(7,-2),(6,2),(8,-2)]$

Table 6. Explicit bounded truant sets computed by the batched algorithm.

Table 6 is the first genuine arithmetic output of the note: explicit bounded truant profiles for several real-quadratic lattices. The second row corresponds to the classical ternary lattice

$$\langle 1, 1, (5+\sqrt{5})/2\rangle,$$

written in the basis $\{1, \omega_5\}$ as coefficients $[(1,0),(1,0),(2,1)]$; the absence of bounded truants up to trace bound 40 is consistent with the known universality picture over $\mathbb{Q}(\sqrt{5})$ [Kala 2023]. The $D = 13$ row is especially compact and illustrates the usefulness of the batched method when the coefficient list contains repetitions. The bundled notebook reproduces this table, the balancing example above, and the optimisation diagnostics from Table 4.

For an explicit omission witness, take $D = 21$, $L = \langle (2,1),(1,0),(1,0)\rangle$, and $\alpha = (4,2)$, which is the first truant in the last row of Table 6. Here

$$\mathcal{S}_{(1,0)}(\alpha) = \varnothing, \qquad \mathcal{S}_{(2,1)}(\alpha) = \{(2,1)\}.$$

In the internally reordered coefficient order $[1,2,0]$, the value lists are therefore

$$V_1(\alpha) = V_2(\alpha) = \{0\}, \qquad V_3(\alpha) = \{0,(2,1)\}.$$

Hence the dynamic-programming state sets are

$$P_1(\alpha) = \{0\}, \qquad P_2(\alpha) = \{0\}, \qquad P_3(\alpha) = \{0,(2,1)\},$$

so $\alpha \notin P_3(\alpha)$ and therefore $\alpha \nrightarrow L$. This is a typical explicit non-representability certificate produced by the same infrastructure.

8 Concluding remarks

The paper is deliberately positioned as a computational number-theory contribution rather than as a classification paper. Its novelty lies in the exact real-quadratic specialisation and integration of weighted lattice enumeration, constructive diagonal representability, coefficient-order optimisation, and batched bounded search. The arithmetic examples above show that the infrastructure already yields explicit bounded truant data.

Several natural extensions suggest themselves. First, one can replace the diagonal weighted-value sets by bounded sections of more general positive definite lattices and ask for a batched bounded search for non-diagonal forms. Second, extending the framework to non-maximal orders would require a careful treatment of the conductor and modified coordinate models, but the present exact comparison machinery should still be relevant there. Third, extending beyond real quadratic fields would force a qualitative change in the geometry: in degree $d > 2$ the weighted trace form becomes a positive-definite lattice problem in rank d, so for cubic fields the search region is an ellipsoid in $\mathbb{R}^3$ and the exact row-interval formulas and canonical half-ellipse scan used here no longer suffice. Fourth, the current routines are designed to be used as a subroutine in diagonal escalator and criterion-set computations of the type considered by Kala–Krásenský–Romeo [Kala et al. 2024]. Finally, the present DP and meet-in-the-middle state-count bounds are purely combinatorial; it would be interesting to sharpen them by exploiting additional multiplicative structure inside O_D^+.

References

Ulrich Fincke and Michael E. Pohst. 1985. Improved methods for calculating vectors of short length in a lattice, including a complexity analysis. *Math. Comp.* 44, 170 (1985), 463–471.

Peter M. Gruber and Cornelis G. Lekkerkerker. 1987. *Geometry of Numbers* (2 ed.). North-Holland, Amsterdam.

Vítězslav Kala. 2023. Universal quadratic forms and indecomposables in number fields. *Communications in Mathematics* 31, 2 (2023), 81–114.

Vítězslav Kala, Kristýna Kramer, and Jakub Krásenský. 2025. Kitaoka's conjecture and sums of squares. arXiv:2510.19545 [math.NT]

Vítězslav Kala, Jakub Krásenský, and Gianluca Romeo. 2024. Universality criterion sets for quadratic forms over number fields. arXiv:2410.22507 [math.NT]

Tomasz Kania. 2026. quadratic_sos: Exact integral lengths of sums of squares in real quadratic rings. Software repository. https://github.com/tomaszkania/quadratic_sos

Przemysław Koprowski. 2022. Solving sums of squares in global fields. In *Proceedings of the 2022 International Symposium on Symbolic and Algebraic Computation (ISSAC '22)*. Association for Computing Machinery, New York, NY, USA, 319–324.

Przemysław Koprowski. 2026. Isotropic vectors over global fields. *Math. Comp.* 95, 359 (2026), 1541–1567. doi:10.1090/mcom/4165

Przemysław Koprowski and Alfred Czogała. 2018. Computing with quadratic forms over number fields. *Journal of Symbolic Computation* 89 (2018), 129–145.

Przemysław Koprowski and Bernd Rothkegel. 2023. The anisotropic part of a quadratic form over a number field. *Journal of Symbolic Computation* 115 (2023), 39–52.

Matěj Raška. 2023. Representing multiples of m in real quadratic fields as sums of squares. *Journal of Number Theory* 244 (2023), 24–41.